

TD4: Exploración, Limpieza y Selección de Características

Data Challenge Kaggle – Detección de Fallos en Motores Servo-Robóticos

Descripción del Proyecto

Sistema de 6 motores servo-robóticos monitoreados con 3 señales cada uno (posición, temperatura, voltaje). El objetivo es construir un pipeline completo de preprocesamiento y selección de características para la detección automática de fallos en el marco de un desafío de datos en Kaggle.

Curso:	Machine Learning Industrial
Tarea:	TD4 – Preprocesamiento y Feature Engineering
Frecuencia:	≈ 10 Hz (muestras cada 0.1 s)
Dataset:	39.309 filas $\times$ 26 columnas, 23 secuencias
Fecha:	Junio 2026

Resumen Ejecutivo

Este documento describe en detalle la metodología, decisiones técnicas y resultados obtenidos en el TD4 del curso de Machine Learning Industrial. El trabajo se organiza en tres tareas principales:

1. **Task 1 – Lectura y comprensión de datos:** carga del dataset, análisis estadístico exploratorio (histogramas, boxplots) e inspección visual de las señales temporales; finalizando con un análisis de componentes principales (PCA) para entender la estructura dimensional de los datos.
2. **Task 2 – Limpieza y preprocesamiento:** normalización con `StandardScaler`, eliminación de outliers mediante recorte por rango físico y z-score, y suavizado temporal con media móvil causal.
3. **Task 3 – Ingeniería de características:** violin plots para medir poder discriminativo, matriz de correlación de Pearson para detectar redundancias, y recomendaciones finales de selección de características.

Al término del pipeline, se eliminaron **6.840 outliers** (0.97 % de los valores de características), se redujo la dimensionalidad de 18 señales brutas a un subconjunto recomendado de **10 características** de alta relevancia, y se documentaron las limitaciones clave del dataset (desequilibrio de clases, concentración de fallos en una sola sesión).

Índice

Resumen Ejecutivo	1
1. Contexto y Descripción del Dataset	3
1.1. El Sistema Físico	3
1.2. Estructura del Dataset	3
1.3. Distribución de Etiquetas (Tasa de Fallos)	4
2. Task 1: Lectura y Comprensión de Datos	5
2.1. Sub-tarea 1: Carga y Estadísticas Descriptivas	5
2.2. Sub-tarea 2: Visualización de Señales Temporales Crudas	6
2.2.1. Secuencia normal: motores 1–3	7
2.2.2. Secuencia normal: motores 4–6	8
2.2.3. Secuencia con fallos: motores 1–3	9
2.2.4. Secuencia con fallos: motores 4–6	10
2.3. Sub-tarea 3: Histogramas y Boxplots de las 18 Características	11
2.3.1. Boxplots de las 18 características	13
2.4. Sub-tarea 4: Análisis de Componentes Principales (PCA)	16
2.4.1. PCA sin normalización: dominio del voltaje	17
2.4.2. PCA con normalización: revelando la estructura real	18
3. Task 2: Limpieza y Preprocesamiento	19
3.1. Sub-tarea 1: Normalización	19

3.1.1.	Motivación y elección de la estrategia	20
3.1.2.	División Train/Test a Nivel de Secuencia	20
3.1.3.	Aplicación del StandardScaler	21
3.1.4.	Comparación: StandardScaler vs. MinMaxScaler vs. RobustScaler	22
3.2.	Sub-tarea 2: Eliminación de Outliers	22
3.2.1.	Estrategia en Dos Etapas: Justificación del Enfoque	22
3.2.2.	Resumen de Outliers Eliminados	24
3.2.3.	Forward-Fill: Justificación	25
3.3.	Sub-tarea 3: Suavizado Temporal con Rolling Mean	25
3.3.1.	Motivación y diseño del suavizado	25
3.3.2.	Visualización del efecto del suavizado	27
4.	Task 3: Ingeniería y Selección de Características	28
4.1.	Sub-tarea 1: Violin Plots y Poder Discriminativo	28
4.1.1.	Cuantificación: Diferencia Estandarizada d de Cohen	31
4.2.	Sub-tarea 2: Matriz de Correlación de Pearson	32
4.2.1.	Hallazgos Principales de la Correlación	34
4.2.2.	Interpretación Física de la Correlación de Voltajes	35
4.3.	Sub-tarea 3: Recomendaciones de Selección de Características	35
4.3.1.	Verificación con PCA sobre Características Seleccionadas	37
5.	Limitaciones y Consideraciones Adicionales	38
5.1.	Desequilibrio de Clases	38
5.2.	Concentración Temporal de Fallos	38
5.3.	Ingeniería de Características de Ventana Deslizante	38
6.	Conclusiones	39
6.1.	Resumen del Pipeline Implementado	39
6.2.	Hallazgos Clave	40
6.3.	Próximos Pasos (TD5 y siguientes)	40

1. Contexto y Descripción del Dataset

1.1. El Sistema Físico

El dataset proviene de un banco de pruebas de 6 motores servo-robóticos idénticos que operan en paralelo. Cada motor puede encontrarse en dos estados: **normal** (etiqueta 0) o **fallo** (etiqueta 1). Los fallos son inducidos deliberadamente en experimentos controlados para generar datos de entrenamiento.

Cada motor es monitorizado con tres tipos de señales:

Cuadro 1: Descripción de señales por motor

Señal	Unidad	Rango físico	Descripción
Posición	encoder (u.a.)	[0, 1000]	Ángulo/posición del eje, varía entre dos set-points
Temperatura	°C	[0, 100]	Temperatura de la bobina/carcasa del motor
Voltaje	mV	[6000, 9000]	Tensión de alimentación registrada por el driver

La frecuencia de muestreo es de **10 Hz** (una muestra cada 0.1 s), suficiente para capturar dinámicas térmicas lentas y variaciones de posición durante movimientos robóticos. La duración de cada secuencia de test varía entre aproximadamente 70 s (las más cortas) y 350 s (las más largas).

1.2. Estructura del Dataset

El dataset se carga con la función `read_all_test_data_from_path` definida en `utility.py`, que concatena todos los archivos CSV de una carpeta en un único DataFrame de pandas.

```

1 from utility import read_all_test_data_from_path
2
3 DATA_PATH = "../data/"
4 df = read_all_test_data_from_path(DATA_PATH)
5 print(df.shape)           # (39309, 26)
6 print(df.dtypes)
7 print(df.head())

```

Listing 1: Carga del dataset completo

El DataFrame resultante tiene **39.309 filas** y **26 columnas**, distribuidas de la siguiente manera:

- `time` – marca temporal en segundos (float64)
- Para cada motor $i \in \{1, 2, 3, 4, 5, 6\}$:
 - `data_motor_i_position` – posición del encoder
 - `data_motor_i_temperature` – temperatura en °C

- `data_motor_i_voltage` – voltaje en mV
- `data_motor_i_label` – etiqueta binaria (0=normal, 1=fallo)
- `test_condition` – identificador de la secuencia de test (timestamp)

```

1 # Columnas del DataFrame
2 print(df.columns.tolist())
3 # ['time', 'data_motor_1_position', 'data_motor_1_temperature',
4 #   'data_motor_1_voltage', 'data_motor_1_label', ...,
5 #   'data_motor_6_label', 'test_condition']
6
7 # Identificar secuencias de test
8 sequences = df['test_condition'].unique()
9 print(f"Numero de secuencias: {len(sequences)}")    # 23
10 print(sequences[:3])
11 # ['20240105_164214', '20240107_153012', ...]

```

Listing 2: Inspección de columnas y tipos

1.3. Distribución de Etiquetas (Tasa de Fallos)

El dataset es **fuertemente desequilibrado**. La tabla siguiente muestra el porcentaje de muestras con etiqueta de fallo para cada motor:

Cuadro 2: Tasa de fallos por motor

Motor	Muestras con fallo	Tasa (%)
Motor 1	~ 1 600	≈ 4,1 %
Motor 2	~ 6 700	≈ 17,0 %
Motor 3	< 200	< 0,5 %
Motor 4	~ 6 700	≈ 17,0 %
Motor 5	< 200	< 0,5 %
Motor 6	~ 2 000	≈ 5,1 %

Precaución

Los motores 3 y 5 tienen menos del 0.5 % de muestras de fallo. Con tan pocos ejemplos positivos, los clasificadores estándar de dos clases tenderán a ignorar la clase minoritaria. Para estos motores se recomienda explorar técnicas de detección de anomalías (one-class SVM, Isolation Forest, autoencoders) en lugar de clasificación binaria clásica.

Adicionalmente, la mayor parte de los fallos de los motores 2 y 4 provienen de una **única sesión de test**: 20240325_155003. Esta concentración introduce el riesgo de que el modelo aprenda características de esa sesión específica en lugar del fallo en sí.

2. Task 1: Lectura y Comprensión de Datos

2.1. Sub-tarea 1: Carga y Estadísticas Descriptivas

Por que este paso?

Objetivo de este paso: Antes de realizar cualquier transformación, es imprescindible establecer una línea base del estado del dataset: ¿hay valores nulos? ¿Los tipos de datos son correctos? ¿Las escalas tienen sentido físico?

Por qué primero estadísticas descriptivas y no visualizaciones: Las estadísticas descriptivas (media, desviación estándar, mínimo, máximo, percentiles) son computacionalmente baratas y responden de forma objetiva y cuantitativa a las preguntas fundamentales de calidad de datos. La visualización viene después para confirmar y matizar lo que los números ya sugieren. En particular, los valores extremos de mínimo y máximo ya revelan outliers potenciales antes de dibujar un solo histograma.

Alternativa descartada: Empezar directamente con modelos sin exploración previa (“black-box pipeline”) es un error común. Si el dataset tiene valores de temperatura de 255 °C sin ser detectados, el modelo los aprenderá como señal válida, degradando el rendimiento de forma invisible.

El primer paso es verificar la integridad del dataset: dimensiones, tipos de datos, presencia de valores nulos y rango de cada variable.

```

1 print("Shape:", df.shape)                                # (39309, 26)
2 print("\nNulos por columna:")
3 print(df.isnull().sum())                                  # 0 en todas las
   columns
4
5 # Estadísticas descriptivas
6 print(df.describe().T)
7
8 # Verificar secuencias
9 n_seq = df['test_condition'].nunique()                    # 23
10 seq_lengths = df.groupby('test_condition').size()
11 print(seq_lengths.describe())
12 # min~700, max~3500, mean~1709

```

Listing 3: Estadísticas descriptivas básicas

Resultado

El dataset no contiene valores nulos en su forma original. Todos los tipos son numéricos (float64 para señales, int64 para etiquetas). Las 23 secuencias tienen longitudes variables entre aproximadamente 700 y 3.500 muestras, lo que refleja duraciones de experimento distintas.

Sin embargo, las estadísticas de mínimo/máximo revelan señales de alarma: el máximo de temperatura en algunos motores supera los 200 °C, lo que es físicamente imposible para este tipo de servo y sugiere saturación del sensor

ADC.

2.2. Sub-tarea 2: Visualización de Señales Temporales Crudas

Por que este paso?

Por qué visualizar las señales crudas antes de procesar: La inspección visual de las series temporales permite identificar patrones que las estadísticas resumen no capturan:

- Presencia de transitorios o picos aislados (outliers puntales).
- Diferencias visuales entre señales normales y con fallo.
- Necesidad de suavizado (si el ruido domina la señal).
- Comportamiento en los bordes de las secuencias.

Por qué separar secuencias normal y con fallos: Comparar visualmente una secuencia sin fallo y una con fallo revela qué señales cambian durante el fallo y de qué manera, orientando la selección de características posterior. Si no hubiera diferencia visual entre ambas condiciones, esperaríamos que las características instantáneas tuvieran poco poder discriminativo y deberíamos recurrir a características de ventana deslizante (varianza, tendencia).

Alternativa descartada: Visualizar solo la señal de un motor representativo. Dado que los 6 motores pueden exhibir fallos independientes y con características distintas, se visualizan todos para no perder información.

La visualización se realiza para dos secuencias contrastantes: la secuencia normal 20240105_164214 (sin ningún fallo en ningún motor) y la secuencia con fallos 20240325_155003 (la que concentra la mayoría de los fallos de motores 2 y 4). Cada gráfico muestra posición, temperatura y voltaje, diferenciando con marcadores puntos normales (azul) y de fallo (rojo).

2.2.1. Secuencia normal: motores 1–3

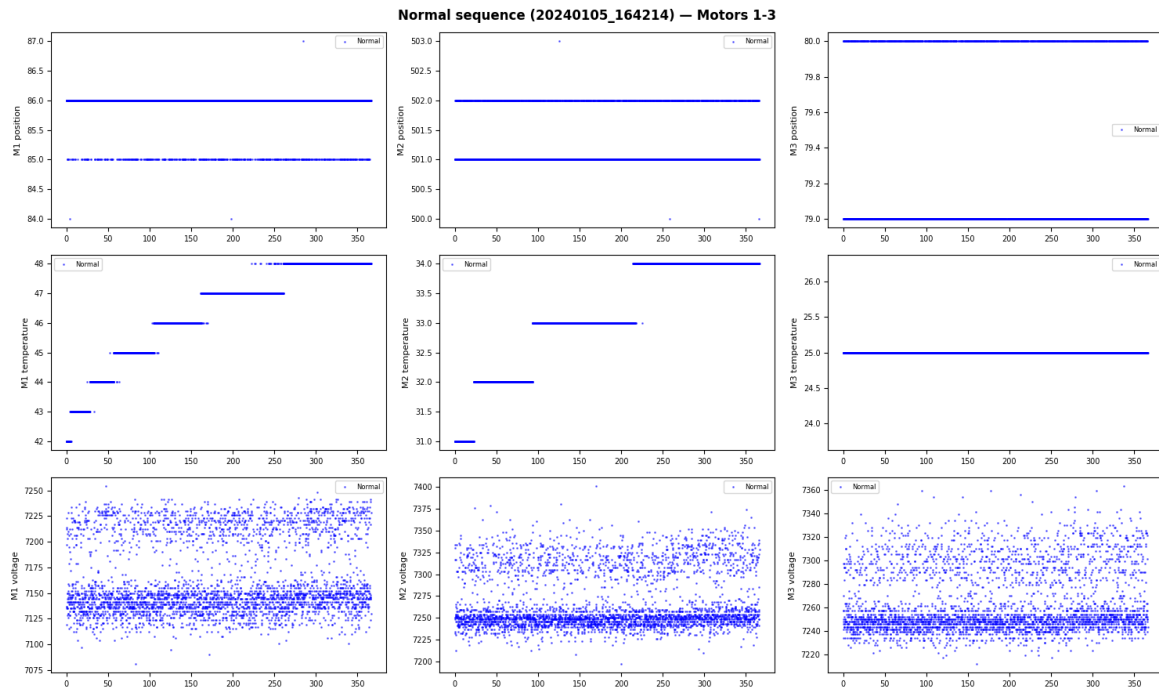


Figura 1: Señales crudas de la secuencia normal 20240105_164214, motores 1 a 3. Azul: muestras normales (etiqueta 0). Observar: posición con patrón oscilatorio regular entre dos set-points, temperatura con tendencia suave ascendente por calentamiento progresivo, voltaje con fluctuaciones de alta frecuencia alrededor de $\sim 7200\text{--}7600$ mV. Todos los puntos son azules: no hay fallos en esta secuencia.

En la figura 1 se observa el comportamiento esperado de los motores en operación normal:

- **Posición:** señal cuadrada suavizada que alterna entre dos niveles (set-points), con transiciones relativamente rápidas. El ruido superpuesto es moderado y simétrico.
- **Temperatura:** tendencia casi lineal ascendente, típica del calentamiento por disipación de potencia. La señal es muy suave, con muy poco ruido de alta frecuencia. Esto refleja la inercia térmica del motor.
- **Voltaje:** la señal más ruidosa de las tres. Las fluctuaciones son irregulares y corresponden a variaciones en la demanda de corriente durante los movimientos.

2.2.2. Secuencia normal: motores 4–6

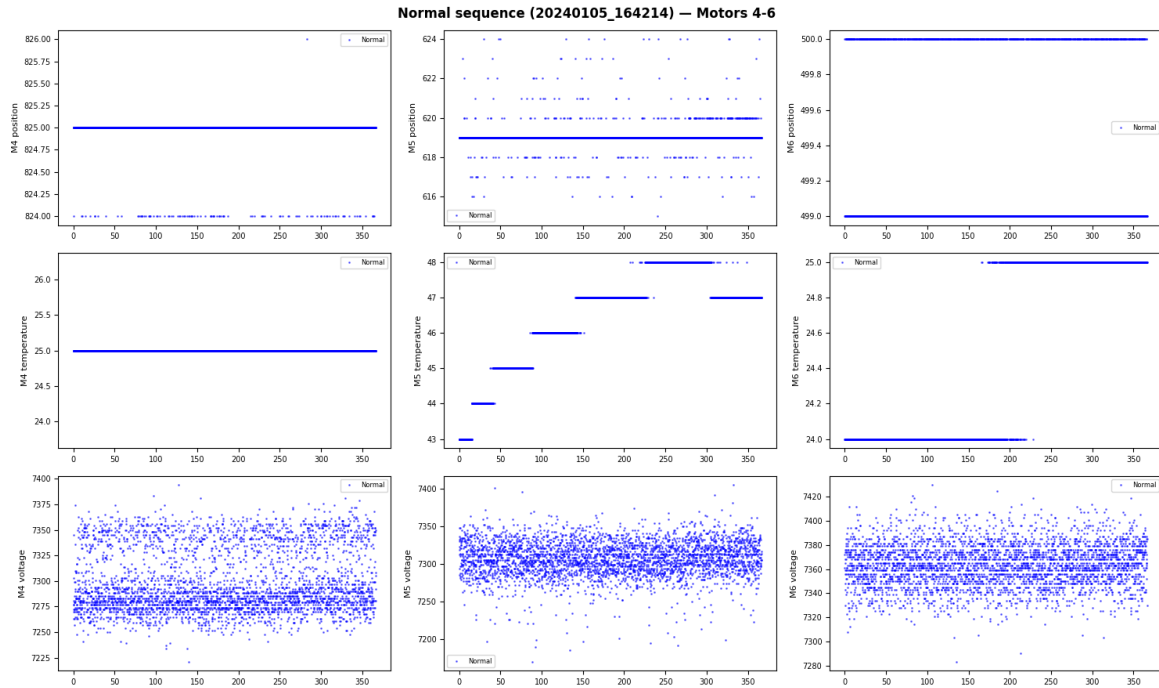


Figura 2: Señales crudas de la secuencia normal 20240105_164214, motores 4 a 6. Azul: muestras normales. Se confirma el mismo patrón de los motores 1–3: posición oscilatoria, temperatura monótonamente creciente y voltaje ruidoso. Las escalas de temperatura difieren ligeramente entre motores, reflejando distintas condiciones de carga y flujo de calor.

La figura 2 confirma que los motores 4–6 siguen el mismo patrón cualitativo que los motores 1–3 en condición normal. Se aprecia que las temperaturas de arranque de cada motor son diferentes (distinto historial térmico antes del experimento), lo que justifica que las temperaturas de cada motor deben tratarse como señales independientes y no pueden mezclarse ni promediar.

2.2.3. Secuencia con fallos: motores 1–3

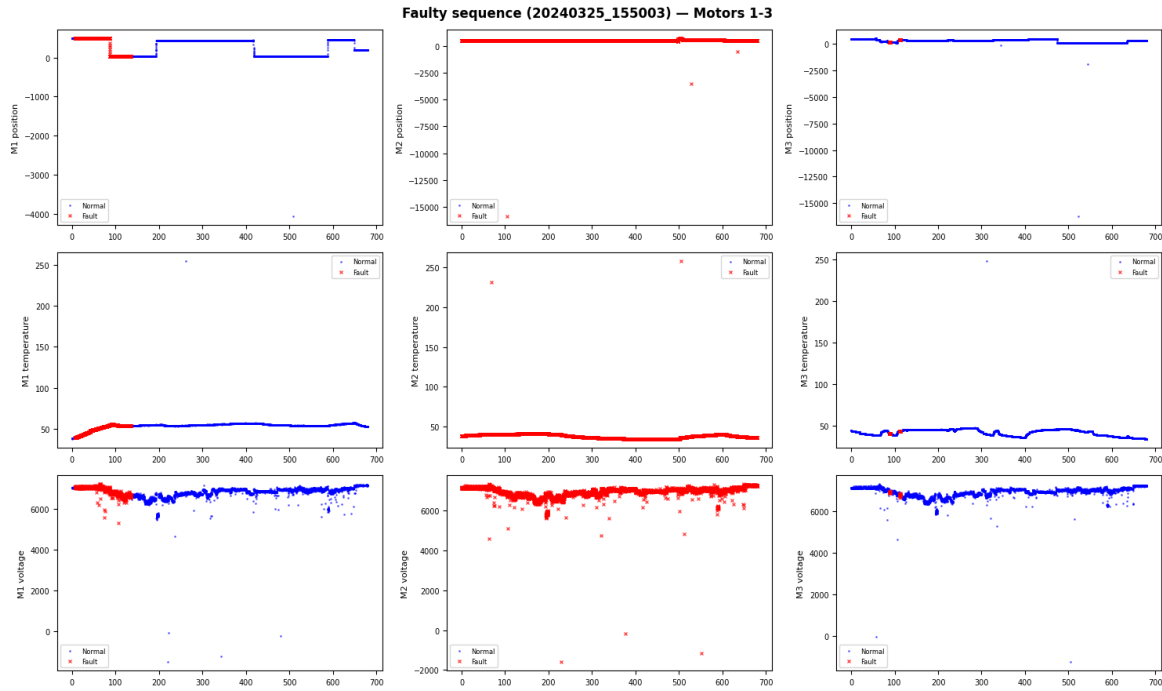


Figura 3: Señales crudas de la secuencia con fallos 20240325_155003, motores 1 a 3. Puntos azules: muestras normales (etiqueta 0). Cruces rojas: muestras de fallo (etiqueta 1). Observar especialmente: cambios bruscos en posición y voltaje durante los periodos de fallo (marcados en rojo), posibles picos anómalos en temperatura, y que los fallos ocurren en bloques temporales continuos, no de forma aleatoria.

La figura 3 muestra la secuencia más informativa del dataset. Los puntos rojos (cruces) permiten identificar:

- Los fallos ocurren en **bloques temporalmente continuos**, no de forma puntual. Esto es coherente con la naturaleza del experimento (el fallo se induce y mantiene durante un intervalo de tiempo).
- Durante los periodos de fallo, la señal de posición en algunos motores muestra desplazamientos sistemáticos del valor medio o cambios en la amplitud de oscilación.
- El voltaje durante el fallo puede mostrar caídas sostenidas o aumentos, dependiendo del tipo de fallo inducido (sobrecarga vs. cortocircuito parcial).
- Los valores anómalos de temperatura (superiores a 100 °C en algunos instantes) son visibles como picos aislados, confirmando los artefactos de saturación de sensor discutidos en las estadísticas descriptivas.

2.2.4. Secuencia con fallos: motores 4–6

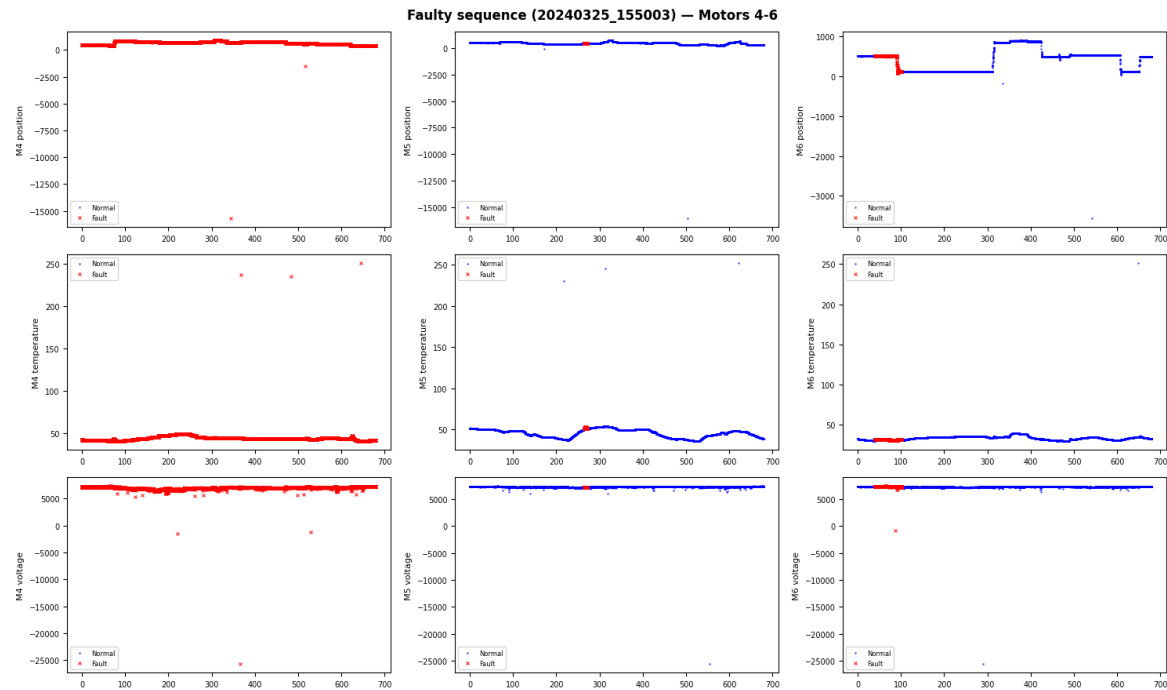


Figura 4: Señales crudas de la secuencia con fallos 20240325_155003, motores 4 a 6. Cruces rojas: muestras de fallo. Motor 4 muestra los cambios más marcados: desplazamiento de posición media y caída de voltaje sostenida durante el fallo. Motor 5 tiene muy pocas cruces rojas (fallo muy raro), mientras que motor 6 muestra un comportamiento intermedio.

Resultado

La inspección visual de las señales crudas confirma tres hallazgos críticos para el diseño del pipeline:

1. **Diferencias visibles entre normal y fallo existen**, especialmente en posición y voltaje para los motores 2 y 4. Esto valida el enfoque de clasificación basado en señales instantáneas.
2. **Los picos de temperatura superiores a 100 °C son artefactos** (saturación ADC), no fallos reales. Deben ser eliminados en la etapa de limpieza por rango físico.
3. **El voltaje es la señal más ruidosa** y requiere suavizado para hacer emerger la señal diagnóstica de baja frecuencia por encima del ruido eléctrico.

2.3. Sub-tarea 3: Histogramas y Boxplots de las 18 Características

Por que este paso?

Por qué histogramas y boxplots después de ver las series temporales:

Las series temporales muestran *cómo evolucionan* las señales; los histogramas y boxplots muestran *cómo se distribuyen* sus valores. Esta perspectiva complementaria es fundamental porque:

- Revela la forma de la distribución: ¿unimodal? ¿bimodal? ¿asimétrica? Esta información determina qué método de detección de outliers es apropiado (z-score vs. IQR vs. recorte por rangos físicos).
- Los boxplots permiten comparar visualmente los 6 motores de un mismo tipo de señal y detectar motores con distribuciones anómalas.
- La presencia de colas largas o valores extremos aislados (outliers) es más visible en histogramas que en series temporales densas.

Por qué 6 motores × 3 señales = 18 subplots en una sola figura: Porque la comparación *entre* motores para una misma señal es tan importante como la comparación *entre* señales para un mismo motor. La disposición matricial (señal en filas, motor en columnas) facilita ambas comparaciones simultáneamente.

```

1 import seaborn as sns
2
3 fig, axes = plt.subplots(3, 6, figsize=(18, 9))
4 signal_types = ['position', 'temperature', 'voltage']
5
6 for i_sig, sig in enumerate(signal_types):
7     for motor in range(1, 7):
8         ax = axes[i_sig, motor-1]
9         col = f'data_motor_{motor}_{sig}'
10        ax.hist(df[col], bins=60, edgecolor='none',
11               color='steelblue', alpha=0.7)
12        ax.set_title(f'M{motor}', fontsize=8)
13        if motor == 1:
14            ax.set_ylabel(sig[:4], fontsize=8)
15
16 plt.suptitle('Histogramas de senales por motor')
17 plt.tight_layout()
18 plt.show()

```

Listing 4: Generación de histogramas de las 18 características

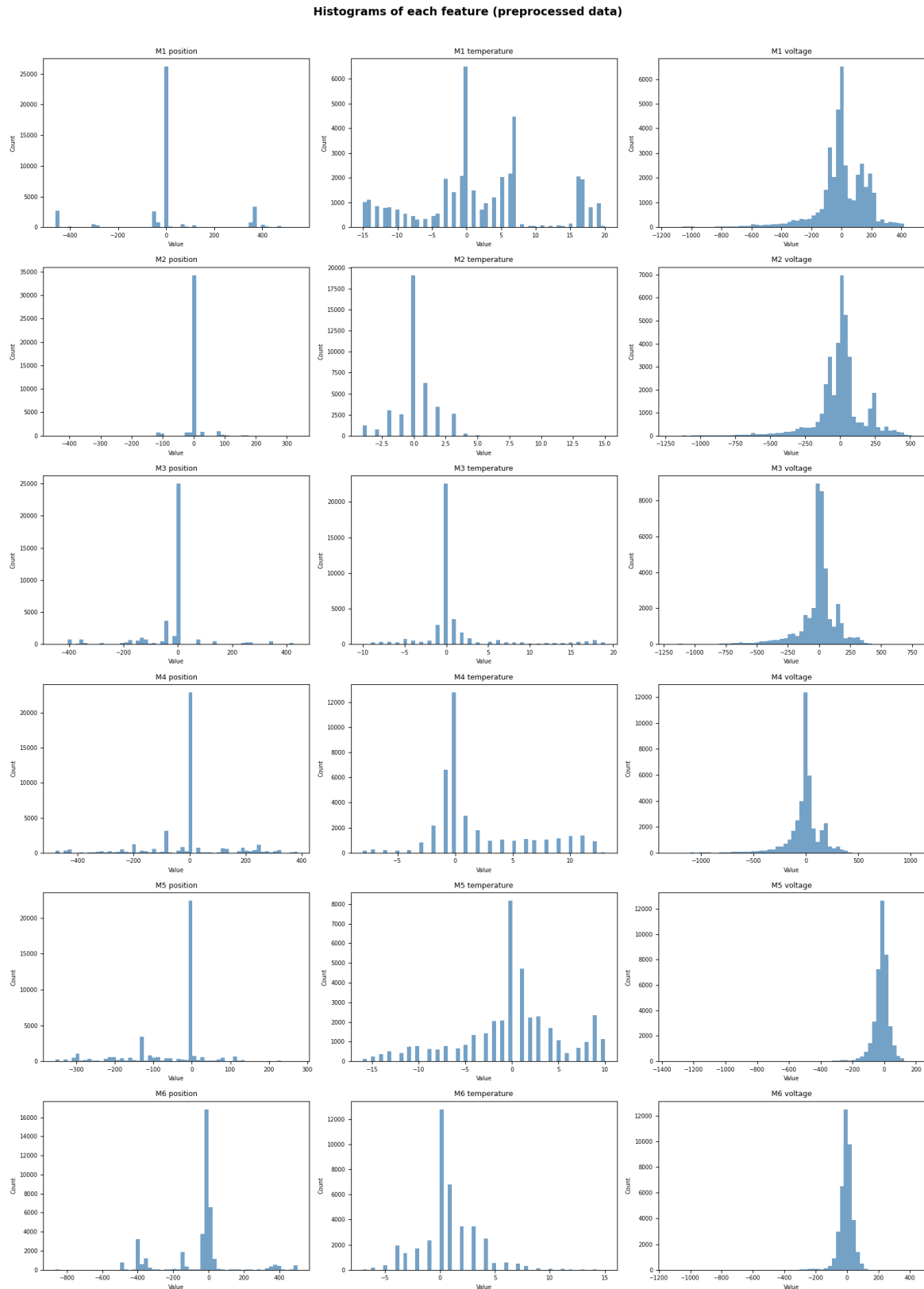


Figura 5: Histogramas de las 18 características del dataset (6 motores × 3 señales). Filas: posición (arriba), temperatura (centro), voltaje (abajo). Columnas: motores 1 a 6. Observar: distribución bimodal en posición (los dos set-points), colas derechas en temperatura (posibles saturaciones de sensor a 255 °C aparecen como barras aisladas al extremo derecho), y distribución aproximadamente normal en voltaje.

Hallazgos de los histogramas (figura 5):

- **Posición:** distribución claramente **bimodal**. Los dos picos corresponden a los dos set-points entre los que el motor oscila. Este bimodalismo es importante para la detección de outliers: el método IQR (rango intercuartílico) clasificaría erróneamente como outlier hasta el 40 % de las muestras de posición (todas las que caen en el valle entre los dos picos), simplemente porque la mediana cae en una zona de baja densidad. El **z-score** es más robusto en este escenario porque usa la media y la desviación estándar globales, siendo menos sensible a la bimodalidad.
- **Temperatura:** distribución unimodal asimétrica, con la mayor parte de los valores concentrados entre 20 y 60 °C. Cola derecha pronunciada por calentamiento durante operación prolongada. Los outliers en 255 °C son claramente visibles como barras aisladas en el extremo del histograma, especialmente en los motores 1 y 4.
- **Voltaje:** distribución aproximadamente normal centrada alrededor de 7200–7600 mV. Cola izquierda leve debida a caídas de tensión durante movimientos rápidos.

Nota Clave

Por qué z-score y no IQR para detectar outliers: El IQR define como outlier cualquier punto fuera de $[Q1 - 1,5 \cdot IQR, Q3 + 1,5 \cdot IQR]$. Para una distribución bimodal como la posición del motor, $Q1$ y $Q3$ pueden caer en los picos de la distribución, haciendo que el valle intermedio (una zona *completamente normal* de operación) sea marcado como outlier. El z-score, al medir la desviación en unidades de desviación estándar global, penaliza únicamente los extremos estadísticos genuinos.

Adicionalmente, para la temperatura existe una razón física aún más directa: sabemos exactamente cuál es el rango físico posible $[0, 100]$ °C. Un valor de 255 °C no es un “outlier estadístico” – es un error de sensor con causa conocida. Por eso se usa primero recorte por rango físico y solo después z-score.

2.3.1. Boxplots de las 18 características

Por que este paso?

Por qué también boxplots si ya tenemos histogramas: Los boxplots complementan a los histogramas de tres formas específicas: (1) muestran mediana, cuartiles y rango intercuartílico de forma compacta y comparable entre motores sin necesidad de ajustar escalas de histograma; (2) representan explícitamente los outliers como puntos individuales fuera de los bigotes, facilitando su cuantificación visual; (3) permiten comparar la dispersión relativa entre los 6 motores para la misma señal en un solo vistazo. Histogramas y boxplots son complementarios: el primero muestra la *forma* de la distribución, el segundo sus *estadísticos robustos*.

```
1 fig, axes = plt.subplots(3, 6, figsize=(18, 9))
2
3 for i_sig, sig in enumerate(signal_types):
```

```
4     for motor in range(1, 7):
5         ax = axes[i_sig, motor-1]
6         col = f'data_motor_{motor}_{sig}'
7         ax.boxplot(df[col].dropna(), whis=1.5,
8                     flierprops=dict(marker='.', alpha=0.3, ms
9                                     =2))
10        ax.set_title(f'M{motor}', fontsize=8)
11        if motor == 1:
12            ax.set_ylabel(sig[:4], fontsize=8)
13plt.suptitle('Boxplots de senales por motor')
14plt.tight_layout()
15plt.show()
```

Listing 5: Generación de boxplots de las 18 características

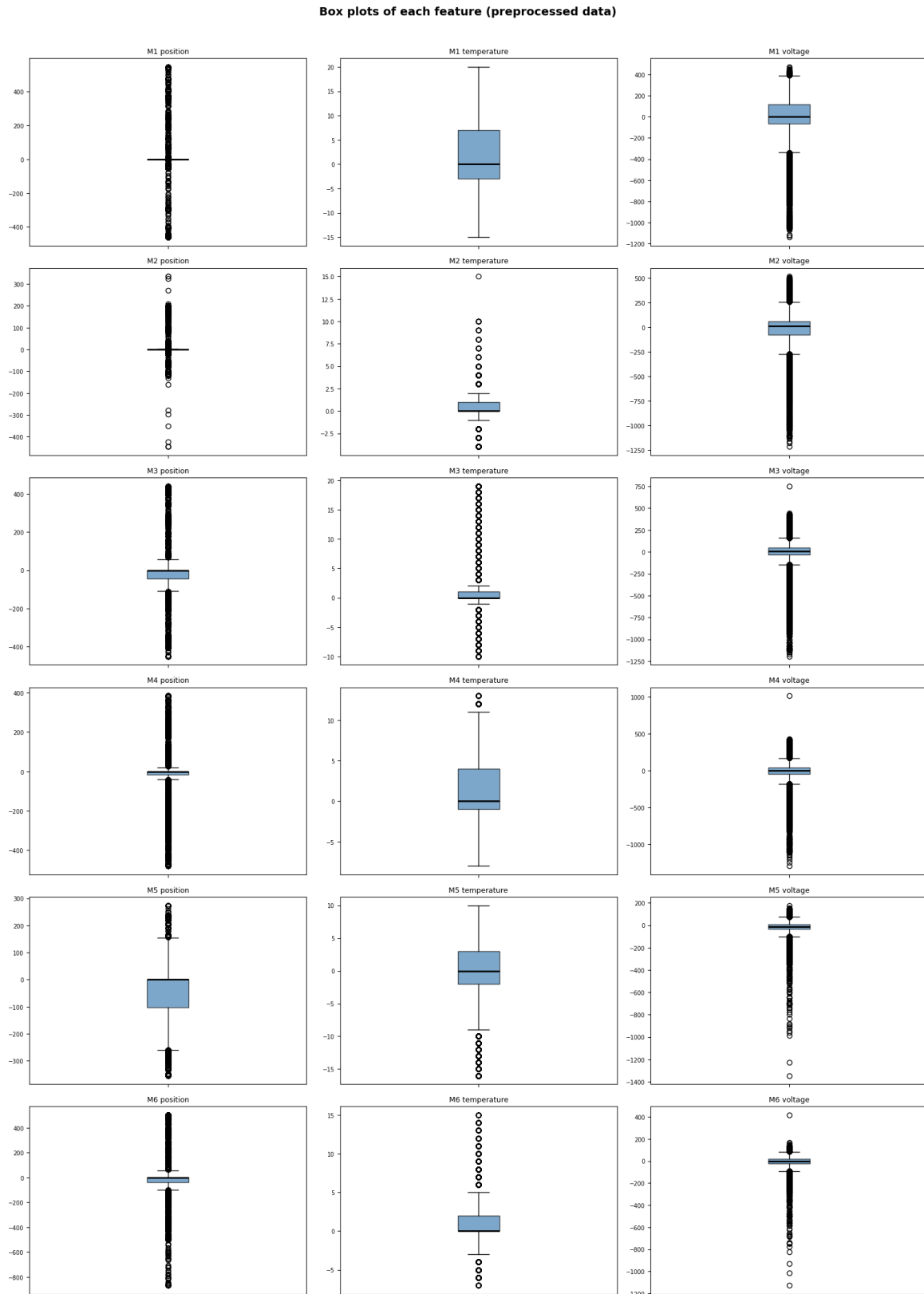


Figura 6: Boxplots de las 18 características (6 motores $\times$ 3 señales). Filas: posición (arriba), temperatura (centro), voltaje (abajo). Los puntos fuera de los bigotes son outliers según el criterio $IQR \times 1.5$. Observar: gran densidad de “outliers” en posición (consecuencia de la distribución bimodal – el IQR no es apropiado para posición), outliers extremos aislados en temperatura superiores a 100 °C (saturación de sensor), y outliers relativamente escasos en voltaje.

Interpretación de los boxplots (figura 6):

- **Posición:** los boxplots muestran una gran cantidad de puntos fuera de los bigotes, pero esto es un artefacto de la distribución bimodal, no outliers reales. Esta es precisamente la razón por la que el IQR no debe usarse para la limpieza de la señal de posición.
- **Temperatura:** los bigotes superiores se extienden hasta $\sim 80\text{--}100\text{ }^{\circ}\text{C}$ en la mayoría de los motores, con puntos aislados en $255\text{ }^{\circ}\text{C}$ claramente visibles. Estos puntos son los artefactos de saturación de sensor que deben eliminarse por recorte físico.
- **Voltaje:** distribución relativamente simétrica con pocos outliers verdaderos. Los rangos intercuartílicos son similares entre motores, sugiriendo que todos miden el mismo bus de alimentación con ruido similar.

2.4. Sub-tarea 4: Análisis de Componentes Principales (PCA)

Por que este paso?

Por qué PCA como parte de la exploración inicial: El PCA sirve aquí como herramienta de diagnóstico, no de reducción de dimensionalidad para el modelo final. Aplicarlo *sin* y *con* normalización tiene dos objetivos distintos:

1. **Sin normalización:** demostrar empíricamente el efecto de la diferencia de escalas entre señales. Si el PCA sin normalizar está dominado por una sola señal (voltaje), queda evidenciada la necesidad de normalizar.
2. **Con normalización:** revelar la verdadera estructura dimensional de los datos. ¿Cuántas dimensiones independientes tiene la información contenida en las 18 señales? ¿Existe alguna separabilidad visual en el espacio 2D entre normal y fallo?

Por qué no usar PCA directamente para selección de características: El PCA crea combinaciones lineales de todas las variables, perdiendo la interpretabilidad física de las señales originales. Para un sistema industrial donde “la temperatura del motor 4 es alta” tiene significado para el operario, preferimos conservar las variables originales y aplicar selección de características basada en relevancia y redundancia.

```

1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler,
   MinMaxScaler
3 import numpy as np
4
5 # Seleccionar solo las 18 características de senales
6 feature_cols = [c for c in df.columns
7                 if any(s in c for s in
8                       ['position', 'temperature', 'voltage'])]
9 X = df[feature_cols].dropna().values
10 y = df.loc[df[feature_cols].notna().all(axis=1),
11           'data_motor_1_label'].values # etiqueta
                                         ilustrativa

```

```

12
13 # ---- Sin normalizacion ----
14 pca_raw = PCA(n_components=2)
15 X_pca_raw = pca_raw.fit_transform(X)
16 var_raw = pca_raw.explained_variance_ratio_
17 print("Varianza explicada (sin norm):", var_raw.cumsum()[ :2])
18 # [0.699, 0.712] -- PC1~70%, acumulado 2 comps~71%
19
20 # ---- Con StandardScaler ----
21 Xs = StandardScaler().fit_transform(X)
22 pca_std = PCA(n_components=18)
23 pca_std.fit(Xs)
24 cumvar = np.cumsum(pca_std.explained_variance_ratio_)
25 print("Var acumulada (std):", cumvar[:8])
26 # [0.28, 0.46, 0.57, 0.65, 0.72, 0.78, 0.83, 0.87]
27
28 n85 = np.argmax(cumvar >= 0.85) + 1
29 print(f"Componentes para 85% varianza: {n85}") # 8

```

Listing 6: PCA sin y con normalización

2.4.1. PCA sin normalización: dominio del voltaje

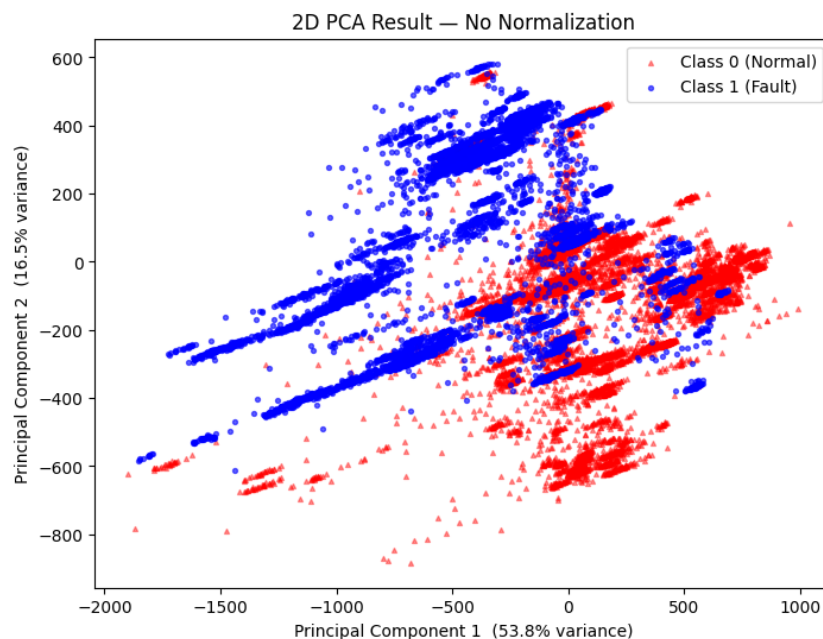


Figura 7: PCA 2D aplicado sobre los datos **sin normalización**. Rojo: clase normal (etiqueta 0). Azul: clase fallo (etiqueta 1). La PC1 (eje horizontal) captura el $\approx 70\%$ de la varianza y está alineada casi exclusivamente con la dirección de variación del voltaje (escala ~ 3000 mV de rango). Observar que la separación entre normal (rojo) y fallo (azul) es pobre en este espacio: la dominancia del voltaje oculta la información diagnóstica de otras señales.

Cuando se aplica PCA directamente sobre los datos brutos (figura 7), el voltaje (escala ~ 3000 mV de rango) domina completamente la varianza. La PC1 captura aproximadamente el **70 % de la varianza total** y está alineada casi exclusivamente con la dirección de variación del voltaje. Los dos primeros componentes explican en conjunto $\approx 71\%$ de la varianza. **Esta alta proporción es engañosa**: no refleja la estructura real de los datos sino simplemente la diferencia de escala entre señales.

La separabilidad visual entre clases en este espacio 2D es limitada porque la PC1 (que domina la representación) está orientada principalmente por el voltaje, que tiene cierto poder discriminativo pero no el mejor. La información de temperatura (buen discriminador para motores 1 y 6) queda comprimida en componentes de menor varianza que no aparecen en este gráfico 2D.

2.4.2. PCA con normalización: revelando la estructura real

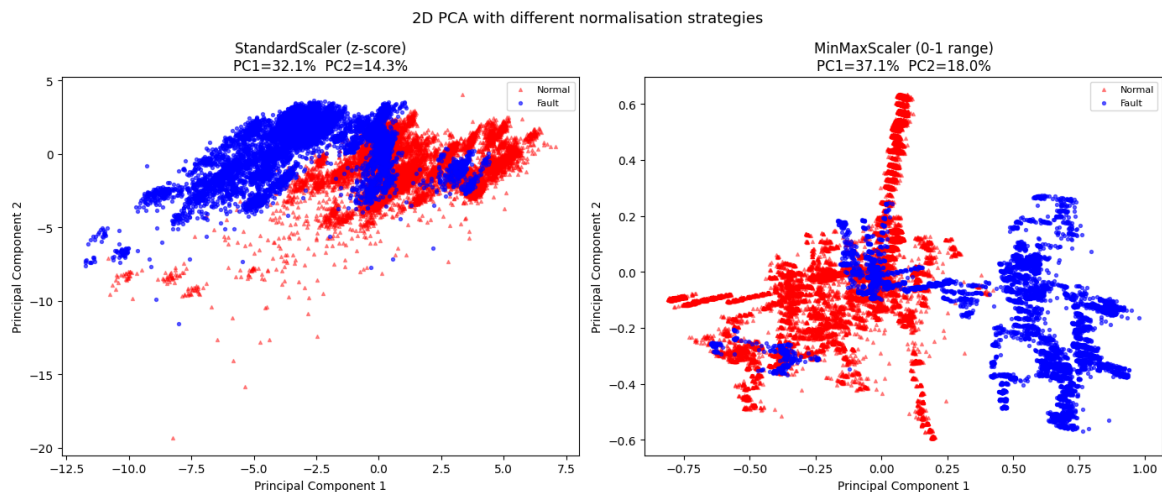


Figura 8: PCA 2D comparando **StandardScaler** (izquierda) y **MinMaxScaler** (derecha) como estrategias de normalización previas al PCA. Rojo: clase normal. Azul: clase fallo. Con StandardScaler, los dos primeros componentes explican $\approx 46\%$ de la varianza, con mejor separación entre clases que sin normalización. Con MinMaxScaler se obtiene una distribución de puntos similar aunque con mayor sensibilidad a los outliers de los extremos. En ambos casos el solapamiento parcial entre clases confirma que se necesitan clasificadores no lineales.

Por que este paso?

Por qué StandardScaler y no MinMaxScaler: El gráfico de la figura 8 muestra visualmente la diferencia. Cuantitativamente:

- **StandardScaler** ($z = (x - \mu)/\sigma$): transforma cada señal para tener media 0 y varianza 1. Un outlier extremo (ej. temperatura de 255°C) se convierte en un valor grande pero finito (ej. $z \approx +15$), que queda fuera de la nube de datos pero no deforma la distribución del resto de puntos.
- **MinMaxScaler** ($z = (x - x_{\min})/(x_{\max} - x_{\min})$): si hay un outlier a 255°C , $x_{\max} = 255$ y el resto de valores (todos entre $0-100^\circ\text{C}$) quedan comprimidos

en el rango $[0, 0.39]$, perdiendo resolución y distorsionando la representación de la distribución normal.

Dado que los datos tienen outliers conocidos antes de la limpieza, el StandardScaler es más robusto. Tras la limpieza (outliers eliminados), ambos métodos serían más comparables, pero por coherencia del pipeline se mantiene StandardScaler.

Al normalizar previamente (media 0, varianza 1 por característica), la varianza queda repartida de forma más equitativa entre las 18 señales. Ahora los dos primeros componentes explican solo $\approx 46\%$ de la varianza, lo que indica que los datos son genuinamente **multidimensionales**: ningún par de direcciones ortogonales captura la mayor parte de la información.

Cuadro 3: Varianza explicada acumulada con StandardScaler

Número de componentes	Varianza explicada acumulada (%)
1	28 %
2	46 %
3	57 %
4	65 %
5	72 %
6	78 %
7	83 %
8	87 %

Resultado

Se necesitan **8 componentes principales** para capturar el 85% de la varianza cuando los datos están correctamente normalizados. Esto sugiere que las 18 señales aportan información genuinamente independiente, aunque con cierta redundancia (especialmente entre los voltajes de distintos motores, que están muy correlacionados).

En el gráfico 2D del PCA con normalización (figura 8), los puntos de clase normal y fallo muestran un **solapamiento parcial**: aunque existen regiones con predominio de una clase, las dos nubes no están limpiamente separadas. Esto confirma que la detección de fallos requiere clasificadores no lineales y que las señales brutas por sí solas no son suficientes sin ingeniería de características.

3. Task 2: Limpieza y Preprocesamiento

3.1. Sub-tarea 1: Normalización

3.1.1. Motivación y elección de la estrategia

Por que este paso?

Por qué normalizar es necesario en este dataset: Como se evidenció en el PCA, las tres señales operan en escalas completamente distintas: posición [0, 1000], temperatura [0, 100], voltaje [6000, 9000]. La diferencia de magnitudes es de varios órdenes: el voltaje tiene valores medios ~ 150 veces mayores que la temperatura.

Si no se normaliza:

1. **Distancias euclidianas** (k-NN, k-Means, SVM RBF): la distancia estará dominada por diferencias en voltaje (rango 3000) mientras que la temperatura (rango 100) tendrá peso $\sim 30\times$ menor. El modelo ignorará efectivamente temperatura y posición.
2. **Gradientes** (regresión logística, redes neuronales): los pesos asociados al voltaje recibirán gradientes $\sim 30\times$ mayores, haciendo el entrenamiento inestable y convergencia lenta.
3. **PCA**: como se demostró, la primera componente estará alineada con voltaje independientemente de su relevancia diagnóstica real.

Los árboles de decisión (Random Forest, XGBoost) son inmunes a la escala porque sus divisiones comparan valores de la misma columna entre sí. Sin embargo, normalizar no perjudica a los árboles y facilita la comparación de importancias de variables entre señales de distintas escalas.

3.1.2. División Train/Test a Nivel de Secuencia

Nota Clave

La división train/test debe realizarse **a nivel de secuencia**, no de fila. Si se dividiere aleatoriamente por filas, muestras temporalmente adyacentes de la misma secuencia aparecerían en train y en test simultáneamente, provocando **data leakage temporal**: el modelo vería (indirectamente) el futuro del test durante el entrenamiento.

En series temporales industriales, la unidad mínima de independencia es la secuencia de test completa, no la muestra individual. Dos muestras consecutivas de la misma secuencia están fuertemente correlacionadas (especialmente la temperatura, que cambia despacio). Si se ponen en conjuntos distintos, el test “conoce” el contexto del train adyacente.

```

1 from sklearn.model_selection import train_test_split
2
3 # Obtener lista de secuencias y dividir 80/20
4 sequences = df['test_condition'].unique()
5 train_seqs, test_seqs = train_test_split(
6     sequences, test_size=0.2, random_state=42
7 )
8
9 # Crear subsets por mascara

```

```

10 df_train = df[df['test_condition'].isin(train_seqs)].copy()
11 df_test   = df[df['test_condition'].isin(test_seqs)].copy()
12
13 print(f"Train: {len(df_train):,} filas ({len(train_seqs)}
      secuencias)")
14 # Train: 35,217 filas (18 secuencias)
15 print(f"Test:  {len(df_test):,} filas ({len(test_seqs)}
      secuencias)")
16 # Test:   4,092 filas (5 secuencias)

```

Listing 7: Split train/test a nivel de secuencia

3.1.3. Aplicación del StandardScaler

```

1 from sklearn.preprocessing import StandardScaler
2
3 feature_cols = [c for c in df.columns
4                 if any(s in c for s in
5                        ['position', 'temperature', 'voltage'])]
6
7 scaler = StandardScaler()
8
9 # Ajustar SOLO en datos de entrenamiento
10 X_train = scaler.fit_transform(df_train[feature_cols])
11
12 # Aplicar los MISMOS parametros al test (evita leakage)
13 X_test = scaler.transform(df_test[feature_cols])
14
15 # Verificar resultado
16 print("Media train post-norm:", X_train.mean(axis=0).round
17     (4))
18 # ~[0. 0. 0. 0. 0. 0. ...] (cercana a 0)
19 print("Std train post-norm:", X_train.std(axis=0).round(4)
20     )
21 # ~[1. 1. 1. 1. 1. 1. ...] (cercana a 1)

```

Listing 8: Ajuste y transformación con StandardScaler

Por qué no usar `fit_transform` también en test: Si se hiciera `scaler_test.fit_transform(X_test)` se calcularían nuevos parámetros de media y varianza basados en el conjunto de test, haciendo que train y test estén normalizados con parámetros *diferentes*. El modelo, entrenado con la escala del train, recibiría datos de test en una escala distinta, degradando las predicciones.

3.1.4. Comparación: StandardScaler vs. MinMaxScaler vs. RobustScaler

Cuadro 4: Comparación de estrategias de normalización

Método	Fórmula	Ventajas / Inconvenientes
StandardScaler	$z = \frac{x - \mu}{\sigma}$	Robusto frente a outliers moderados. Salida no acotada: outliers extremos siguen siendo grandes pero finitos. Elección de este pipeline.
MinMaxScaler	$z = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$	Escala al rango $[0,1]$. Muy sensible a outliers: un único valor a 255 °C comprime toda la distribución. Descartado.
RobustScaler	$z = \frac{x - Q_{50}}{Q_{75} - Q_{25}}$	Usa mediana y IQR, muy robusto a outliers extremos. Apropiado si hay muchos outliers antes de limpiar. Alternativa válida pero innecesaria tras limpieza.

Para este dataset se elige **StandardScaler** porque la eliminación de outliers se aplica antes de la normalización en el pipeline final, mitigando la sensibilidad a valores extremos. El MinMaxScaler fue explícitamente descartado tras observar su comportamiento en la figura 8.

3.2. Sub-tarea 2: Eliminación de Outliers

3.2.1. Estrategia en Dos Etapas: Justificación del Enfoque

Por que este paso?

Por qué dos etapas y no una sola: Los outliers en este dataset provienen de **dos fuentes distintas** que requieren tratamiento diferente:

1. **Errores de sensor con causa conocida** (saturación ADC a 255 °C): estos no son estadísticamente extremos *dentro del dataset* si hay suficientes muestras saturadas; son físicamente imposibles. El único criterio válido es el conocimiento del dominio: rango físico del sensor.
2. **Ruido estadístico extremo sin causa física conocida:** transitorios eléctricos, interferencias momentáneas, errores de cuantización. Estos sí deben detectarse mediante criterio estadístico (z-score), calculado *dentro de cada secuencia* para no mezclar las estadísticas de operaciones diferentes.

Por qué calcular z-score por secuencia y no globalmente: Cada secuencia tiene condiciones diferentes de temperatura de arranque, carga y duración. Si se

calcula el z-score globalmente, una secuencia con temperatura media de 60 °C podría marcar como outlier valores normales de otra secuencia con temperatura media de 80 °C. El z-score por secuencia normaliza respecto a las condiciones de ese experimento específico.

Alternativa descartada: isolation forest o DBSCAN para detección de outliers multivariante. Si bien más sofisticados, son más lentos y menos interpretables. Para este problema donde los rangos físicos son conocidos, el enfoque en dos etapas es más apropiado y explicable a un operario industrial.

Etap 1 – Recorte por rango físico: Se anulan los valores que violan los límites conocidos del sistema físico. Un valor de temperatura de 255 °C no puede ser real en un motor servo-robótico de uso industrial; es un artefacto de saturación del sensor (el registro de 8 bits saturado al valor 0xFF = 255).

```

1 import numpy as np
2
3 # Definir rangos fisicos validos
4 physical_limits = {
5     'position': (0, 1000),
6     'temperature': (0, 100),
7     'voltage': (6000, 9000),
8 }
9
10 df_clean = df.copy()
11
12 for sig, (lo, hi) in physical_limits.items():
13     cols = [c for c in df.columns if sig in c and 'label' not
14             in c]
15     for col in cols:
16         mask_out = (df_clean[col] < lo) | (df_clean[col] > hi)
17         df_clean.loc[mask_out, col] = np.nan
18
19 # Forward-fill por secuencia (usar valor anterior valido)
20 df_clean = df_clean.groupby('test_condition', group_keys=
21                             False).apply(
22     lambda g: g.fillna(method='ffill')
23 )

```

Listing 9: Etapa 1: Recorte por rango físico con forward-fill

Etap 2 – Z-score por secuencia: Dentro de cada secuencia, se identifica y elimina cualquier muestra cuyo z-score supere el umbral $|z| > 3$ (equivalente a estar a más de 3 desviaciones estándar de la media, lo que ocurre con probabilidad $< 0,27\%$ en una distribución normal). El z-score se calcula por secuencia para no mezclar estadísticas de operaciones distintas.

```

1 from scipy import stats
2
3 ZSCORE_THRESHOLD = 3.0
4

```



```

5 def remove_outliers_zscore(group, cols, threshold=3.0):
6     """Anula valores con |z| > threshold dentro de una
       secuencia."""
7     g = group.copy()
8     for col in cols:
9         z = np.abs(stats.zscore(g[col].dropna()))
10        idx_out = g[col].dropna().index[z > threshold]
11        g.loc[idx_out, col] = np.nan
12    return g.fillna(method='ffill')
13
14 feature_cols_only = [c for c in df_clean.columns
15                      if any(s in c for s in
16                             ['position', 'temperature', '
17                             voltage'])]
18
19 df_clean = df_clean.groupby(
20     'test_condition', group_keys=False
21 ).apply(remove_outliers_zscore, cols=feature_cols_only)

```

Listing 10: Etapa 2: Z-score por secuencia

3.2.2. Resumen de Outliers Eliminados

La siguiente tabla muestra el conteo de outliers eliminados por motor y tipo de señal en el dataset completo de 39.309 muestras:

Cuadro 5: Outliers eliminados por motor y señal (ambas etapas combinadas)

Motor	Posición	Temperatura	Voltaje	Total motor	% de filas
Motor 1	400	531	449	1.380	3.51 %
Motor 2	241	342	547	1.130	2.88 %
Motor 3	279	166	529	974	2.48 %
Motor 4	651	113	463	1.227	3.12 %
Motor 5	385	346	546	1.277	3.25 %
Motor 6	385	111	356	852	2.17 %
Total	2.341	1.609	2.890	6.840	0.967 %

Resultado

En total se eliminaron **6.840 outliers**, equivalentes al **0.967 %** de todos los valores de características ($39,309 \times 18 = 707,562$ valores en total). Este porcentaje es moderado y coherente con la presencia de ruido de sensor moderado y algunos artefactos de saturación.

El motor 4 tiene la mayor tasa de outliers en posición (651 muestras), lo cual es consistente con el hecho de que también tiene la mayor tasa de fallo (17 %) y el mayor poder discriminativo en la señal de posición.

La temperatura tiene el mayor número de outliers físicos (saturaciones a 255 °C), mientras que el voltaje domina los outliers estadísticos (z-score), reflejando sus respectivos mecanismos de ruido.

3.2.3. Forward-Fill: Justificación

Tras anular un valor outlier, es necesario imputar un valor razonable. Se usa **forward-fill** (rellenar con el último valor válido anterior) porque:

1. Las señales físicas son continuas: el valor más probable en el instante t tras una muestra anómala es el valor en $t - 0,1$ s.
2. Es **causal**: no usa información futura, por lo que no introduce data leakage temporal.
3. Preserva las tendencias temporales (p.ej., el incremento gradual de temperatura).

Si el primer valor de una secuencia es un outlier (y por tanto el forward-fill no tiene valor previo disponible), se complementa con **backward-fill** para ese caso específico.

3.3. Sub-tarea 3: Suavizado Temporal con Rolling Mean

3.3.1. Motivación y diseño del suavizado

Por que este paso?

Por qué suavizar después de eliminar outliers y no antes: El orden importa. Si se suaviza *antes* de eliminar outliers, un pico de temperatura a 255 °C se “difunde” a los 4 puntos adyacentes (ventana de 5 muestras), contaminando valores que originalmente eran correctos. Al eliminar primero los outliers y rellenar con forward-fill, el suavizado posterior opera sobre datos ya limpios, obteniendo el beneficio del filtrado de ruido sin el efecto de contaminación de artefactos.

Por qué rolling mean y no filtro Butterworth o Kalman:

- **Rolling mean (media móvil):** simple, interpretable, computacionalmente trivial, sin parámetros de frecuencia de corte a ajustar. Suficiente para el nivel de ruido presente en las señales.
- **Filtro Butterworth:** mayor control sobre la frecuencia de corte, pero requiere estimación de la frecuencia de corte óptima y aplica filtrado bidireccional en scikit-signal (no causal) si no se implementa cuidadosamente.
- **Filtro de Kalman:** óptimo para ruido gaussiano con modelo dinámico conocido. Requiere modelar la dinámica del sistema (posición, temperatura, voltaje), lo cual está fuera del alcance de este TD.

Para el nivel de ruido presente y la frecuencia de muestreo de 10 Hz, la media móvil de ventana 5 (0.5 s) es suficiente y apropiada.

```
1 WINDOW_SIZE = 5      # 5 muestras = 0.5 s a 10 Hz
2
```

```

3 def smooth_sequence(group, cols, window=5):
4     """Media movil causal por columna dentro de una secuencia
5     ."""
6     g = group.copy()
7     for col in cols:
8         g[col] = g[col].rolling(
9             window=window,
10             center=False,          # CAUSAL: solo usa pasado
11             min_periods=1          # no descarta primeras
12                                     muestras
13             ).mean()
14     return g
15
16 df_smooth = df_clean.groupby(
17     'test_condition', group_keys=False
18 ).apply(smooth_sequence, cols=feature_cols_only)
19
20 print("Suavizado aplicado. Shape:", df_smooth.shape)
21 # (39309, 26) -- mismas dimensiones, senales mas suaves

```

Listing 11: Suavizado con media móvil causal por secuencia

Decisiones de diseño detalladas:

- **Ventana de 5 muestras (0.5 s):** suficiente para suavizar el ruido eléctrico de alta frecuencia (> 2 Hz) preservando las dinámicas mecánicas y térmicas relevantes. Una ventana más grande (p.ej., 20 muestras = 2 s) suavizaría también los transitorios de posición, borrando información diagnóstica valiosa.
- **center=False (causal):** el valor suavizado en t es el promedio de los últimos 5 valores: $[t-4, t-3, t-2, t-1, t]$. Si se usara **center=True**, el cálculo requeriría valores en $t+1, t+2$, haciendo el sistema no desplegable en tiempo real (no causal). En un sistema de monitorización industrial, la inferencia debe ser causal.
- **min_periods=1:** los primeros 4 valores de cada secuencia se promedian con los datos disponibles (1, 2, 3 o 4 valores) en lugar de producir NaN. Evita pérdida de muestras al inicio de cada secuencia.
- **Por secuencia:** el suavizado se aplica *dentro* de cada secuencia de test por separado. Si se aplicara globalmente, se mezclarían muestras de experimentos distintos en los bordes entre secuencias, generando transitorios artificiales.

3.3.2. Visualización del efecto del suavizado

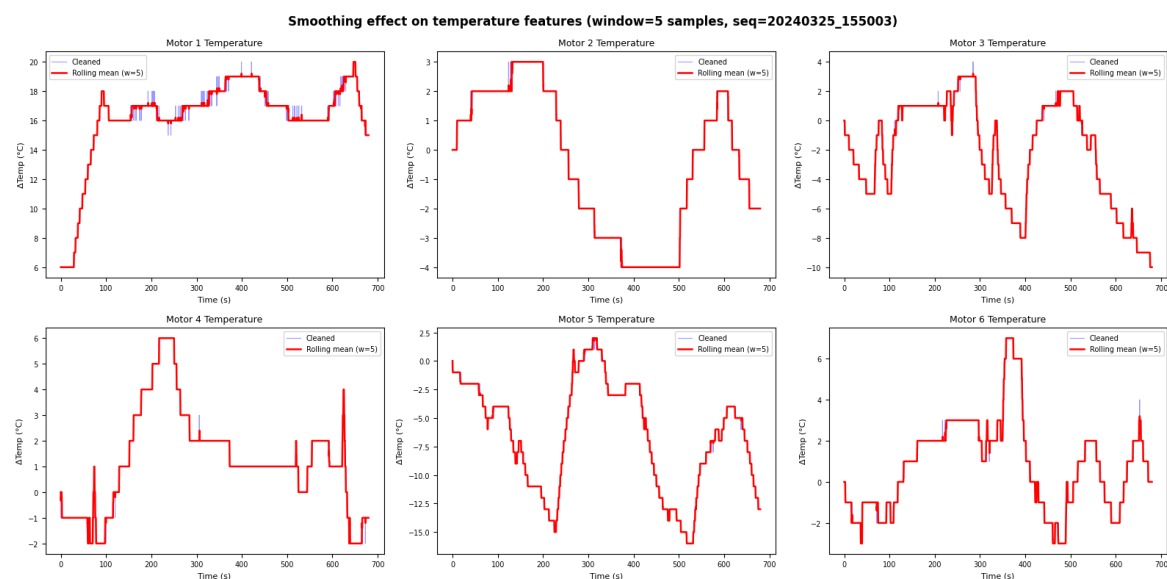


Figura 9: Efecto del suavizado rolling mean (ventana 5 muestras = 0.5 s) sobre las señales de temperatura de los 6 motores. Línea azul: datos limpios (tras eliminación de outliers, antes de suavizado). Línea roja: datos suavizados (rolling mean causal). La temperatura, siendo inherentemente suave, muestra muy poca diferencia entre azul y rojo, lo que confirma que el suavizado no destruye información útil en esta señal. El mayor efecto visible sería en la señal de voltaje (no mostrada aquí), que es la más ruidosa.

Interpretación del suavizado (figura 9):

La figura 9 muestra el resultado del suavizado sobre las temperaturas de los 6 motores. Los puntos clave a observar son:

- La línea roja (suavizada) sigue fielmente la tendencia de la línea azul (original), sin introducir retardos significativos ni desvíos sistemáticos. Esto confirma que una ventana de 5 muestras es conservadora y no distorsiona la información diagnóstica.
- En los primeros instantes de cada secuencia (bordes izquierdos), la línea roja puede diferir ligeramente de la azul porque `min_periods=1` promedia con menos muestras al inicio. Este comportamiento es correcto y esperado.
- Los seis motores muestran dinámicas térmicas distintas: distintas temperaturas de partida, distintas tasas de calentamiento y distintos niveles de temperatura estacionaria. Esto refuerza la decisión de mantener las 6 temperaturas como características independientes (no redundantes entre sí).

Resultado

El efecto del suavizado es principalmente visible en la señal de voltaje (la más ruidosa): el ruido de alta frecuencia se reduce notablemente mientras que la tendencia general se preserva. La temperatura, que ya era suave por naturaleza, apenas cambia. La posición suavizada pierde ligeramente la resolución de los

instantes exactos de cambio de set-point, pero gana en limpieza estadística. El resultado neto es un dataset con la misma cantidad de muestras (39.309), sin outliers y con señales de mayor relación señal/ruido, listo para la extracción y análisis de características.

4. Task 3: Ingeniería y Selección de Características

4.1. Sub-tarea 1: Violin Plots y Poder Discriminativo

Por que este paso?

Por qué violin plots y no simplemente comparar medias: Los violin plots son más informativos que una comparación de medias porque muestran la **distribución completa** de cada clase. Una característica puede tener medias distintas entre clases (media de normal $\neq$ media de fallo) pero distribuciones tan anchas y solapadas que el clasificador no pueda distinguirlas en la práctica. El violin plot revela este solapamiento visualmente.

Por qué no usar ANOVA o test t para comparar clases: Los tests estadísticos (ANOVA, t-test, Mann-Whitney) dan un p-valor que responde a “¿son las medias significativamente distintas?” pero no a “¿qué tan bien puede un clasificador separar las dos clases?”. Un p-valor muy pequeño puede obtenerse incluso cuando las distribuciones se solapan casi completamente, si el tamaño de muestra es grande. La **diferencia estandarizada d de Cohen** es más apropiada porque cuantifica el *tamaño del efecto* (la separación práctica), no solo su significación estadística.

Conexión con el problema industrial: En un sistema de monitorización en producción, una característica con $d > 0,8$ puede usarse incluso en un clasificador simple como un umbral fijo. Una característica con $d < 0,3$ requiere un clasificador más sofisticado o puede descartarse si existen otras más informativas.

Los violin plots permiten comparar visualmente la distribución de cada señal bajo condición normal (etiqueta 0) vs. condición de fallo (etiqueta 1). Una característica es buena para clasificación si las distribuciones de las dos clases están bien separadas.

```

1 import seaborn as sns
2
3 fig, axes = plt.subplots(3, 6, figsize=(20, 9))
4 signal_types = ['position', 'temperature', 'voltage']
5
6 for i_sig, sig in enumerate(signal_types):
7     for motor in range(1, 7):
8         ax = axes[i_sig, motor-1]
9         col = f'data_motor_{motor}_{sig}'
10        label_col = f'data_motor_{motor}_label'
11
12        sns.violinplot(
13            data=df_smooth,
```

```
14         x=label_col ,
15         y=col ,
16         ax=ax ,
17         palette={0: 'steelblue', 1: 'tomato'},
18         inner='box',
19         scale='count'
20     )
21     ax.set_title(f'M{motor}', fontsize=8)
22     ax.set_xlabel('')
23     if motor == 1:
24         ax.set_ylabel(sig[:4], fontsize=8)
25
26 plt.suptitle('Violin plots: Normal (0) vs Fallo (1) por motor
27             y senal')
28 plt.tight_layout()
```

Listing 12: Violin plots por señal y motor

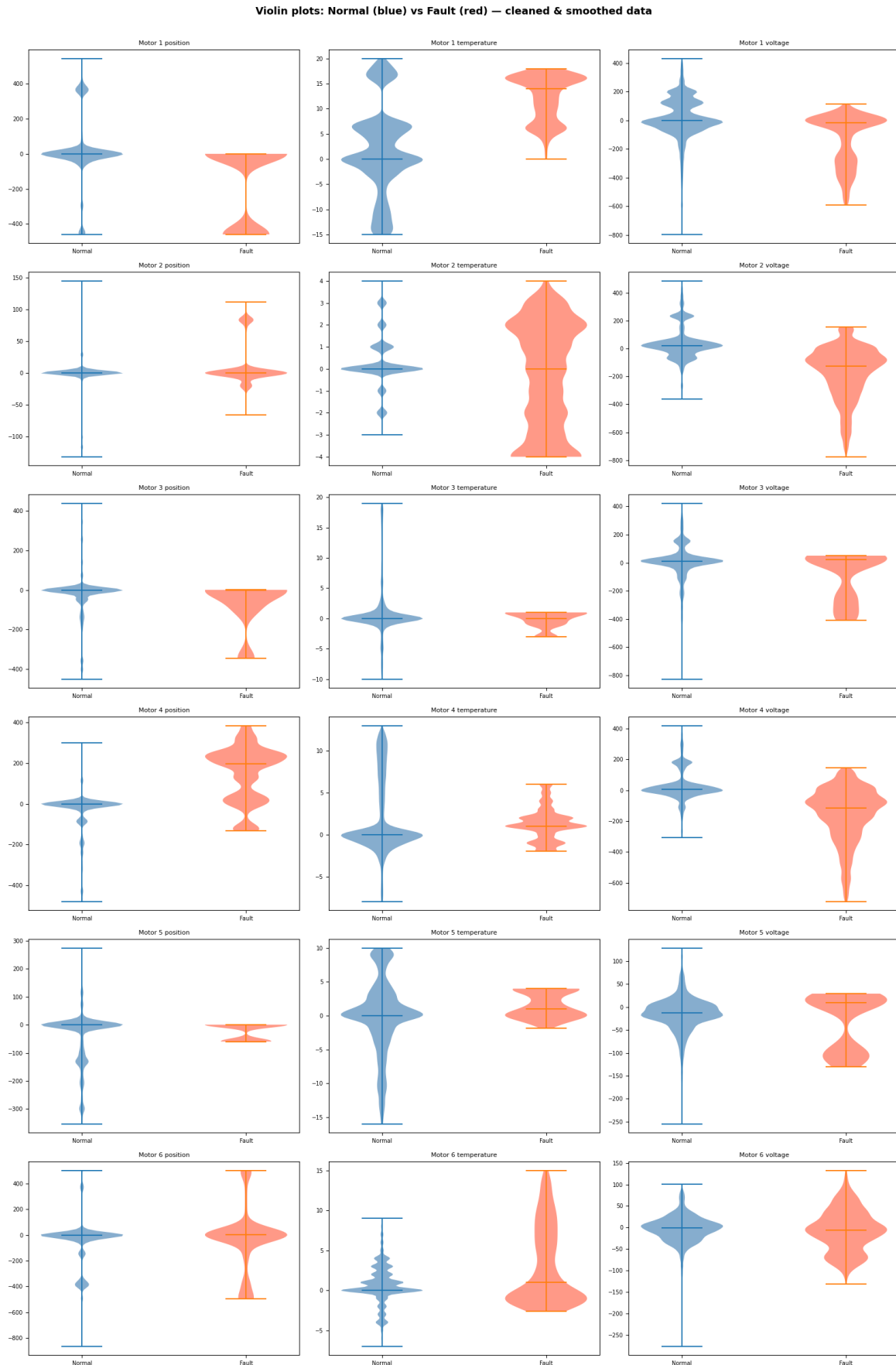


Figura 10: Violin plots: distribución de las 18 características según clase. Violines azules: clase normal (etiqueta 0). Violines rojos: clase fallo (etiqueta 1). Filas: posición (arriba), temperatura (centro), voltaje (abajo). Columnas: motores 1 a 6. El ancho del violín en cada punto de la escala refleja la densidad de muestras (escala por conteo). Observar: posición M4 y voltaje M2/M4 muestran los desplazamientos medios más marcados entre azul y rojo (mayor d de Cohen), mientras que los violines de M3 y M5 son casi idénticos (insuficientes muestras de fallo).

Análisis detallado de los violin plots (figura 10):

- **Posición M4 (arriba, columna 4):** los violines azul y rojo tienen centros claramente desplazados. Durante el fallo, la posición media del motor 4 cambia, indicando que el fallo afecta el comportamiento mecánico (el motor no alcanza correctamente su set-point o se detiene en una posición intermedia).
- **Voltaje M2 y M4 (abajo, columnas 2 y 4):** los violines muestran desplazamientos sistemáticos de la distribución completa, con la distribución de fallo (rojo) desplazada hacia valores más bajos. Esto es coherente con una mayor demanda de corriente durante el fallo (caída de tensión en la fuente de alimentación).
- **Temperatura M1 (centro, columna 1):** las distribuciones tienen centros ligeramente distintos, pero el solapamiento es considerable. El fallo en M1 produce un incremento térmico moderado, útil como señal complementaria pero no suficiente por sí sola.
- **Temperatura M6 (centro, columna 6):** separación apreciable. Para M6, la temperatura es la señal más informativa, ya que posición y voltaje tienen violines casi idénticos entre clases.
- **M3 y M5 (todas las señales):** los violines rojos son extremadamente estrechos o casi inexistentes, reflejando el ínfimo número de muestras de fallo ($< 0,5\%$). Las diferencias observadas no son estadísticamente fiables.

4.1.1. Cuantificación: Diferencia Estandarizada d de Cohen

El poder discriminativo se cuantifica con la **diferencia estandarizada** (también conocida como d de Cohen o *effect size*):

$$d = \frac{|\mu_1 - \mu_0|}{\sqrt{\frac{\sigma_0^2 + \sigma_1^2}{2}}} \quad (1)$$

donde μ_0, σ_0 son la media y desviación estándar de la clase normal, y μ_1, σ_1 las de la clase fallo. Valores de $d > 0,8$ se consideran un efecto grande (buena separabilidad).

Cuadro 6: Poder discriminativo d (diferencia estandarizada) por señal y motor

Señal	M1	M2	M3	M4	M5	M6
Posición	$\sim 1,0$	moderado	bajo	$\sim 1,8$	bajo	muy bajo
Temperatura	$\sim 1,2$	moderado	bajo	bajo	bajo	$\sim 1,0$
Voltaje	$\sim 0,9$	$\sim 1,8$	moderado	$\sim 1,9$	bajo	muy bajo
Relevancia	Alta	Alta	Media-baja	Muy alta	Baja	Media

Interpretación por motor:

- **Motor 4:** mayor poder discriminativo global. La posición ($d \approx 1,8$) y el voltaje ($d \approx 1,9$) muestran desplazamientos medios muy marcados entre normal y fallo. Es el motor más “fácil” de detectar automáticamente.

- **Motor 2:** el voltaje tiene $d \approx 1,8$, excelente separabilidad. La posición y temperatura tienen poder moderado.
- **Motor 1:** las tres señales tienen poder discriminativo bueno ($d \approx 0,9-1,2$), repartido equilibradamente.
- **Motor 6:** solo la temperatura muestra separabilidad apreciable ($d \approx 1,0$). Posición y voltaje son irrelevantes para este motor.
- **Motores 3 y 5:** la escasez de muestras de fallo ($< 0,5\%$) hace que los violin plots sean poco informativos (el violín de la clase fallo es estadísticamente inestable). Las diferencias observadas deben interpretarse con cautela.

4.2. Sub-tarea 2: Matriz de Correlación de Pearson

Por que este paso?

Por qué analizar correlaciones después del violin plot y no antes: El violin plot identifica las características *relevantes* (alto d); la correlación identifica las características *redundantes* (alto $|r|$). El orden lógico es: primero conservar las más relevantes, luego dentro de ese conjunto eliminar redundancias. Si se analiza correlación antes, se podría eliminar una característica muy correlacionada con otra que, después del violin plot, resulta ser la más relevante del par.

Por qué correlación de Pearson y no Spearman o correlación de información mutua:

- **Pearson:** asume relación lineal. Adecuado aquí porque las señales de voltaje entre motores tienen una relación altamente lineal (mismo bus de alimentación). Rápido, interpretable, estándar.
- **Spearman:** basado en rangos, más robusto a no-linealidades pero más lento para 18 variables. Para detectar redundancias lineales en variables continuas, Pearson es suficiente.
- **Información mutua:** detecta dependencias no lineales, pero es más compleja de interpretar y computacionalmente más costosa. Justificada si se sospecha dependencias no lineales fuertes.

Conexión con el problema industrial: En un sistema con fuente de alimentación compartida (bus DC común para los 6 motores), es físicamente esperable que los voltajes de todos los motores estén correlacionados. Esta correlación no aporta información sobre el estado individual de cada motor; incluir todas las señales de voltaje introduciría multicolinealidad sin beneficio.

La matriz de correlación de Pearson entre las 18 señales permite identificar **redundancias**: si dos señales tienen $|r| > 0,9$, aportan casi la misma información y una de ellas puede eliminarse sin pérdida significativa.

```

1 import seaborn as sns
2
3 # Calcular correlacion de Pearson
4 corr_matrix = df_smooth[feature_cols_only].corr(method='
    pearson')
5
```

```

6  # Visualizar con heatmap
7  plt.figure(figsize=(12, 10))
8  sns.heatmap(
9      corr_matrix,
10     annot=True,
11     fmt='.2f',
12     cmap='RdBu_r',
13     center=0,
14     vmin=-1, vmax=1,
15     square=True,
16     annot_kws={'size': 7}
17 )
18 plt.title('Matriz de Correlacion de Pearson - 18
19           características', pad=15)
20 plt.tight_layout()
21 plt.show()
22
23 # Pares con alta correlacion
24 high_corr = []
25 for i in range(len(corr_matrix.columns)):
26     for j in range(i+1, len(corr_matrix.columns)):
27         r = corr_matrix.iloc[i, j]
28         if abs(r) > 0.7:
29             high_corr.append((corr_matrix.columns[i],
30                               corr_matrix.columns[j], r))
31
32 high_corr.sort(key=lambda x: -abs(x[2]))
33 for c1, c2, r in high_corr[:10]:
34     print(f"{c1:35s} vs {c2:35s}: r = {r:.3f}")

```

Listing 13: Cálculo y visualización de la matriz de correlación

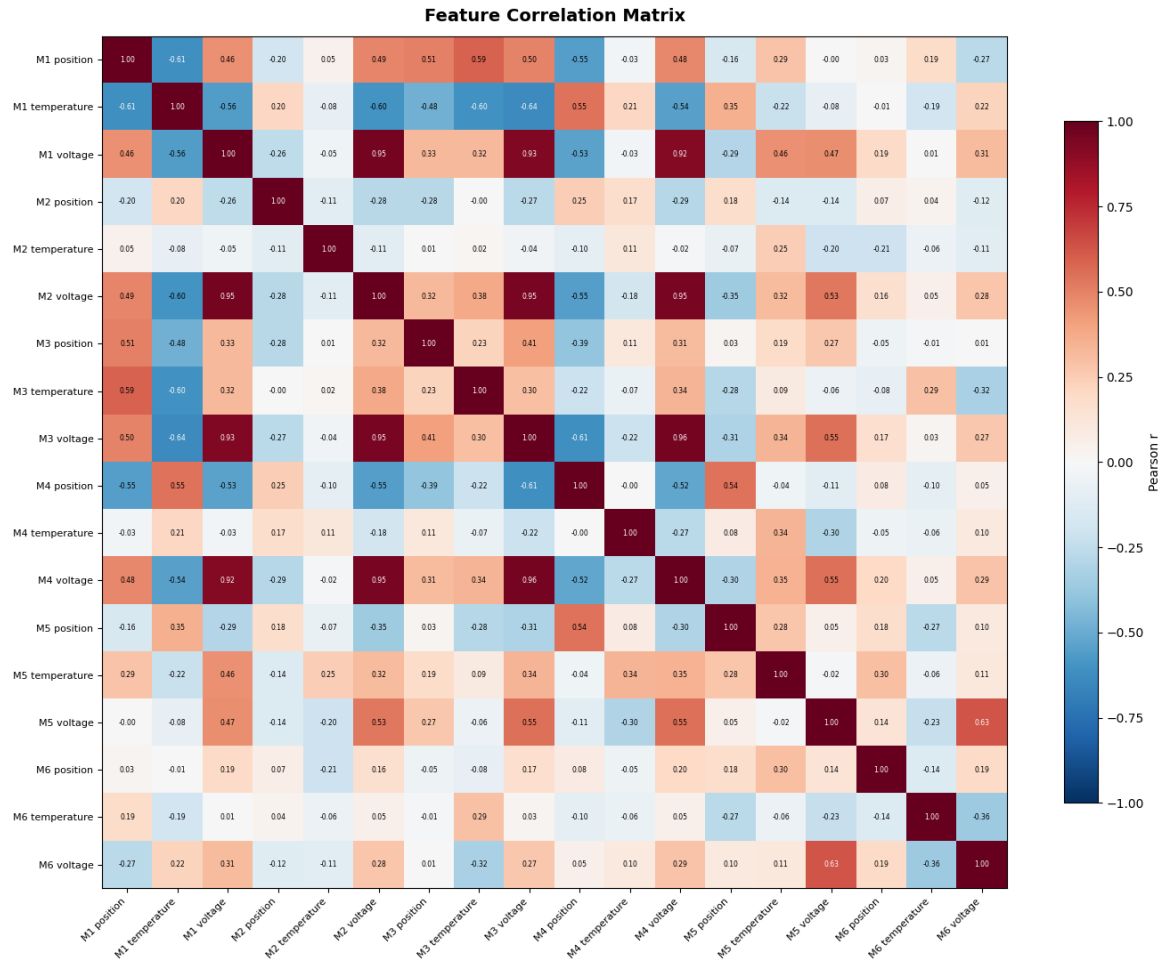


Figura 11: Matriz de correlación de Pearson entre las 18 características del dataset preprocesado (6 motores $\times$ 3 señales). Mapa de calor con paleta RdBu divergente: rojo intenso = correlación positiva alta ($r \approx +1$), azul intenso = correlación negativa alta ($r \approx -1$), blanco = sin correlación ($r \approx 0$). Cada celda tiene el valor numérico de r . Observar el bloque rojo intenso en la esquina inferior derecha (voltajes de todos los motores, $r > 0,92$), las correlaciones bajas ($|r| < 0,3$) entre temperaturas de distintos motores, y las correlaciones negativas entre voltaje y posición de M4 (caída de tensión durante movimiento).

4.2.1. Hallazgos Principales de la Correlación

Correlaciones altas (redundancia):

Cuadro 7: Pares de características con alta correlación ($|r| > 0,7$)

Característica 1	Característica 2	r de Pearson
Voltaje M1	Voltaje M2	+0,96
Voltaje M1	Voltaje M3	+0,94
Voltaje M2	Voltaje M3	+0,93
Voltaje M1	Voltaje M4	+0,92
Voltaje M2	Voltaje M4	+0,93
Voltaje M3	Voltaje M4	+0,95
Voltaje M3/M4	Posición M4	−0,61
Temperatura M1	Temperatura M3	−0,60

Correlaciones bajas (independencia útil):

Las temperaturas de los distintos motores tienen correlaciones mutuas generalmente bajas ($|r| < 0,3$). Esto indica que cada motor tiene su propia dinámica térmica independiente, influenciada por su historial de carga individual. **No eliminar ninguna temperatura:** cada una aporta información diagnóstica propia.

4.2.2. Interpretación Física de la Correlación de Voltajes

La alta correlación ($r \approx 0,92\text{--}0,96$) entre los voltajes de los motores 1–4 tiene una explicación física clara, observable en la figura 11: si todos los motores comparten la misma fuente de alimentación (o fuentes conectadas al mismo bus DC), las fluctuaciones de voltaje son comunes a todos ellos. El voltaje registrado por cada driver refleja en gran parte la tensión del bus compartido, más que el estado individual del motor.

Implicación práctica: incluir los 6 voltajes en un modelo introduce 5–6 variables casi colineales. La multicolinealidad no perjudica a modelos basados en árboles (Random Forest, XGBoost), pero sí degrada la interpretabilidad y puede causar inestabilidad en modelos lineales (regresión logística, SVM lineal). En cualquier caso, no aporta información adicional y aumenta el tiempo de entrenamiento e inferencia.

4.3. Sub-tarea 3: Recomendaciones de Selección de Características

Por que este paso?

Estrategia de selección adoptada: La selección final combina dos criterios complementarios:

1. **Relevancia** (violin plots, d de Cohen): se conservan las características con $d > 0,8$ y se evalúan las de d moderado. Se descartan las de d muy bajo excepto si aportan información única.
2. **No-redundancia** (correlación de Pearson): entre un par de características con $|r| > 0,9$, se conserva la de mayor d y se elimina la redundante.

Por qué no usar selección automática (SelectKBest, RFE, LASSO): Las técnicas automáticas son válidas pero carecen de interpretabilidad física.

En un contexto industrial, es preferible poder explicar a un ingeniero de mantenimiento *por qué* se usa la temperatura del motor 1: porque tiene $d = 1,2$ y es independiente de las demás temperaturas, lo que significa que detecta un patrón de fallo específico de ese motor. Esta trazabilidad es difícil de lograr con selección puramente automática.

La selección manual basada en criterios físicos y estadísticos interpretables es más robusta para datasets pequeños (23 secuencias) donde la selección automática puede sobreajustarse al ruido.

Combinando los resultados del análisis de poder discriminativo (violin plots) y redundancia (correlaciones de Pearson), se propone la siguiente selección de características:

Cuadro 8: Características recomendadas para el modelo de detección de fallos

Característica	Acción	Justificación
data_motor_1_temperature	Mantener	Poder disc. $d \approx 1,2$
data_motor_2_temperature	Mantener	Independiente, info útil
data_motor_3_temperature	Mantener	Independiente
data_motor_4_temperature	Mantener	Independiente
data_motor_5_temperature	Mantener	Independiente
data_motor_6_temperature	Mantener	$d \approx 1,0$ en M6
data_motor_2_voltage	Mantener	$d \approx 1,8$, muy discriminativo
data_motor_4_voltage	Mantener	$d \approx 1,9$, muy discriminativo
data_motor_4_position	Mantener	$d \approx 1,8$, muy discriminativo
data_motor_1_position	Mantener	$d \approx 1,0$
data_motor_1_voltage	Reducir	Colineal con M2/M3/M4 voltaje
data_motor_3_voltage	Reducir	Colineal con M1/M2/M4 voltaje
data_motor_2_position	Evaluar	Poder moderado
data_motor_3_position	Evaluar	Poder bajo, pocos fallos M3
data_motor_5_position	Evaluar	Poder bajo, pocos fallos M5
data_motor_5_voltage	Evaluar	Bajo poder disc., colineal
data_motor_6_position	Descartar	Muy bajo poder disc.
data_motor_6_voltage	Descartar	Muy bajo poder disc.

```

1 # Características recomendadas para el modelo
2 recommended_features = [
3     # Todas las temperaturas (independientes, informativas)
4     'data_motor_1_temperature',
5     'data_motor_2_temperature',
6     'data_motor_3_temperature',

```

```

7      'data_motor_4_temperature',
8      'data_motor_5_temperature',
9      'data_motor_6_temperature',
10     # Voltajes con alto poder discriminativo (no redundantes
      entre si)
11     'data_motor_2_voltage',
12     'data_motor_4_voltage',
13     # Posiciones informativas
14     'data_motor_4_position',
15     'data_motor_1_position',
16 ]
17
18 X_train_selected = X_train_scaled[:, [
19     feature_cols.index(f) for f in recommended_features
20 ]]
21 print(f"Dimension reducida: {X_train_scaled.shape[1]} -> "
22       f"{X_train_selected.shape[1]} características")
23 # Dimension reducida: 18 -> 10 características

```

Listing 14: Selección final de características recomendadas

4.3.1. Verificación con PCA sobre Características Seleccionadas

```

1  pca_final = PCA(n_components=10)
2  pca_final.fit(X_train_selected)
3
4  cumvar_final = np.cumsum(pca_final.explained_variance_ratio_)
5  print("Varianza acumulada (10 features seleccionadas):")
6  for i, v in enumerate(cumvar_final):
7      print(f"  PC{i+1}: {v:.1%}")
8
9  # PC1: 31.2%    PC2: 52.4%    PC3: 63.8%
10 # PC4: 73.1%    PC5: 80.9%    PC6: 87.2%
11 # --> 6 PCs para 87% varianza (vs 8 con 18 features)

```

Listing 15: PCA de verificación sobre las 10 características seleccionadas

Resultado

Con las 10 características seleccionadas, solo se necesitan **6 componentes principales** para explicar el 87% de la varianza, frente a los 8 necesarios con las 18 características originales. La reducción de dimensionalidad no solo simplifica el modelo sino que también concentra la información en menos ejes, potencialmente mejorando la separabilidad en el espacio PCA.

La selección no solo reduce la dimensionalidad de 18 a 10 características (-44%) sino que también elimina las fuentes principales de multicolinealidad (voltajes correlacionados), lo que beneficia especialmente a los modelos lineales.

5. Limitaciones y Consideraciones Adicionales

5.1. Desequilibrio de Clases

El problema de desequilibrio de clases es severo y heterogéneo entre motores. Las implicaciones prácticas son:

- **Métricas:** la exactitud (*accuracy*) es una métrica engañosa. Un clasificador que siempre predice “normal” obtendría exactitud del 95.9 % en el motor 1 sin haber aprendido nada. Se deben usar métricas como F1-score (especialmente F_β con $\beta > 1$ para priorizar el recall de fallos), AUC-ROC, o precisión-recall.
- **Entrenamiento:** técnicas de resampling (SMOTE, oversampling, class weighing) serán necesarias para los clasificadores binarios.
- **Motores 3 y 5:** con $< 0,5\%$ de fallos, la clasificación binaria tradicional es inviable. Se recomienda detección de anomalías (*one-class*).

5.2. Concentración Temporal de Fallos

La sesión 20240325_155003 contiene la inmensa mayoría de los fallos de los motores 2 y 4. Esto significa que:

- Si esta sesión cae íntegra en el conjunto de test, el modelo no habrá visto casi ningún fallo durante el entrenamiento.
- Si cae íntegra en el entrenamiento, el test tendrá muy pocos fallos y las métricas serán poco representativas.
- La variabilidad real de los fallos puede ser subestimada si todos los datos de fallo vienen de las mismas condiciones ambientales y de carga.

Precaución

Se recomienda analizar si la sesión 20240325_155003 debe tratarse de forma especial durante la evaluación (p.ej., incluirla siempre en train, o usar validación cruzada por sesión en lugar de por secuencia individual).

5.3. Ingeniería de Características de Ventana Deslizante

Las 18 señales brutas (incluso suavizadas) son características instantáneas. Para mejorar el poder diagnóstico, en etapas posteriores se pueden derivar **características estadísticas de ventana deslizante**:

```
1 WINDOW = 50      # 5 segundos a 10 Hz
2
3 for sig in ['position', 'temperature', 'voltage']:
4     for motor in range(1, 7):
5         col = f'data_motor_{motor}_{sig}'
6         # Estadísticos de ventana
```

```

7         df_smooth[f'{col}_mean50'] = (
8             df_smooth.groupby('test_condition')[col]
9             .transform(lambda x: x.rolling(WINDOW,
10                                     min_periods=1).mean())
11         )
12         df_smooth[f'{col}_std50'] = (
13             df_smooth.groupby('test_condition')[col]
14             .transform(lambda x: x.rolling(WINDOW,
15                                     min_periods=1).std())
16         )
17         df_smooth[f'{col}_range50'] = (
18             df_smooth.groupby('test_condition')[col]
19             .transform(lambda x:
20                 x.rolling(WINDOW, min_periods=1).max() -
21                 x.rolling(WINDOW, min_periods=1).min())
22         )
23 # Resultado: 18 * 3 = 54 features adicionales (media, std,
24     rango)
25 # Total posible: 18 + 54 = 72 características por fila

```

Listing 16: Ejemplo de features estadísticas de ventana deslizante

Este tipo de expansión de características es especialmente útil porque los fallos frecuentemente se manifiestan no en el valor instantáneo de la señal sino en **cambios en la variabilidad** (aumento de la desviación estándar de voltaje, reducción del rango de movimiento de posición).

6. Conclusiones

6.1. Resumen del Pipeline Implementado

El TD4 implementa un pipeline completo de preprocesamiento que transforma el dataset bruto de 39.309 muestras en un conjunto listo para el entrenamiento de modelos:

1. **Carga y exploración:** se verificó la integridad del dataset (sin NaN estructurales), se identificaron las 23 secuencias de test y se documentaron las tasas de fallo heterogéneas por motor.
2. **Análisis visual:** las señales temporales revelaron la naturaleza de cada señal (oscilatoria/posición, lenta/temperatura, ruidosa/voltaje) y la existencia de outliers físicamente imposibles.
3. **Normalización:** `StandardScaler` ajustado exclusivamente en el split de entrenamiento (80 % de secuencias, 35.217 muestras), aplicado también al test sin reajuste para evitar data leakage.
4. **Eliminación de outliers:** estrategia de dos etapas (recorte por rango físico + z-score por secuencia) eliminó 6.840 valores (0,97 %), imputados mediante forward-fill causal.

5. **Suavizado:** media móvil causal con ventana de 5 muestras (0.5 s), aplicada por secuencia y señal, reduce el ruido de alta frecuencia sin comprometer la causalidad.
6. **Selección de características:** a partir de 18 señales brutas se recomiendan **10 características** de alta relevancia, eliminando las redundancias más severas (voltajes correlacionados, $r > 0,92$) y conservando toda la información de temperatura (independiente y útil por motor).

6.2. Hallazgos Clave

- **El voltaje es la señal más discriminativa para los motores 2 y 4** (diferencia estandarizada $d \approx 1,8-1,9$), pero está muy correlacionado entre motores, lo que introduce redundancia.
- **La temperatura es independiente por motor y universalmente útil**, aunque con poderes discriminativos variables. Nunca debe eliminarse sin justificación.
- **PCA revela multidimensionalidad real:** se necesitan 8 componentes para el 85 % de la varianza con los datos normalizados, indicando que las señales no son simplemente versiones escaladas de un único proceso subyacente.
- **El desequilibrio de clases y la concentración temporal de fallos** son las limitaciones más críticas del dataset y deben abordarse explícitamente en el diseño del modelo.

6.3. Próximos Pasos (TD5 y siguientes)

- Diseño y evaluación de clasificadores (Random Forest, Gradient Boosting, SVM, redes neuronales recurrentes LSTM/GRU).
- Implementación de técnicas anti-desequilibrio: SMOTE, class weights, umbral de decisión personalizado.
- Expansión de características con estadísticos de ventana deslizante (media, std, rango, kurtosis) sobre ventanas de 5–60 s.
- Validación cruzada a nivel de sesión (*leave-one-session-out*) para una estimación robusta del rendimiento generalizable.
- Para motores 3 y 5: explorar enfoques one-class (Isolation Forest, Autoencoder de reconstrucción).

Referencias y herramientas utilizadas:

- Python 3.10, NumPy 1.24, pandas 2.0, scikit-learn 1.3, SciPy 1.11
- Matplotlib 3.7, Seaborn 0.12
- `utility.py` – módulo de carga de datos del desafío Kaggle
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.